



Part of the AgentCon World Tour by the Global AI Community

AI Agents Developer Conference

agentcon.city/rome



Automatizzare la Compliance di Azure Firewall con SRE Agent

Martina Mangione, Elisa Anzelmo



Martina Mangione



Elisa Anzelmo

agentcon.city/rome

Made Possible by



What we'll cover today

01

FIRE architecture

AI agent for firewall compliance on Azure

02

The benchmark

Recall, precision, F1 — real metrics

03

Skill engineering

No fine-tuning, no GPU, no extra cost

04

Live coding

Skill anatomy + agent invocation



The real problem

Azure Firewall in an enterprise tenant

Hierarchical policies

Dozens of parent → child policy chains

Thousands of rules

NetworkRule / ApplicationRule / NatRule entries

Multi-environment

dev / test / prd — all different teams

Weekly changes

From teams that don't talk to each other

Compliance review: email + screenshots + Excel

No structured audit trail

The question:

Can I delegate
the review to an agent?



Audit incoming...



The SecOps engineer the day before audit

Three ways to build the agent

Once you've decided you need an agent, the real question is: how much of the runtime do you build yourself?

DIY — raw model API

- openai.ChatCompletion + your loop
- You write: tool routing, retries, context trimming, fn-call parsing
- Full control · zero abstraction
- ~600 LoC before you call one tool
- Every bug is yours

Agent Framework

- Open SDK (successor of SK + AutoGen)
- Loop, tool registry, multi-agent, planning — out of the box
- Model-agnostic · you pick GPT-5, Grok-4, Llama, DeepSeek...
- You host the runtime (container)
- ~50 LoC to ship

★ BEST CHOICE

SRE Agent (preset)

- Built ON TOP of Foundry/AF
- Verticalized for Azure ops: built-in knowledge, sub-agents, persistent memory
- Microsoft hosts everything
- Skill = .SKILL.md in repo
- ~0 infra
- hours to ship

The Microsoft agent stack

4 layers · Agent Framework + Foundry + Hosted Agents + MCP — complementary, not alternatives

Agent Framework

How the agent REASONS — code-first SDK

- Logic, planning, tool routing, multi-agent
- Model-agnostic · successor of Semantic Kernel + AutoGen
- "Where an agent stops being a prompt and becomes a system"

Microsoft Foundry

Where the agent RUNS — managed runtime

- Prompt · workflow · hosted agent · model catalog (~20)
- Entra identity · App Insights · RBAC · VNET · content safety
- SRE Agent = Microsoft preset built on top of Foundry

Hosted Agents

Zero infra, full control

- Microsoft manages runtime, scaling, isolation
- Built-in persistent memory + state
- You own logic, skill, tools, data

MCP · Model Context Protocol

Standard tools, written ONCE

- Open standard (JSON-RPC) — native Foundry support
- Tool registered 1x → reused by SRE Agent, Claude, Cursor...
- FIRE v2: backend exposed as MCP server, not a custom endpoint

So who actually calls the APIs?

Spoiler: not the LLM. The LLM writes text. The runtime calls the APIs.

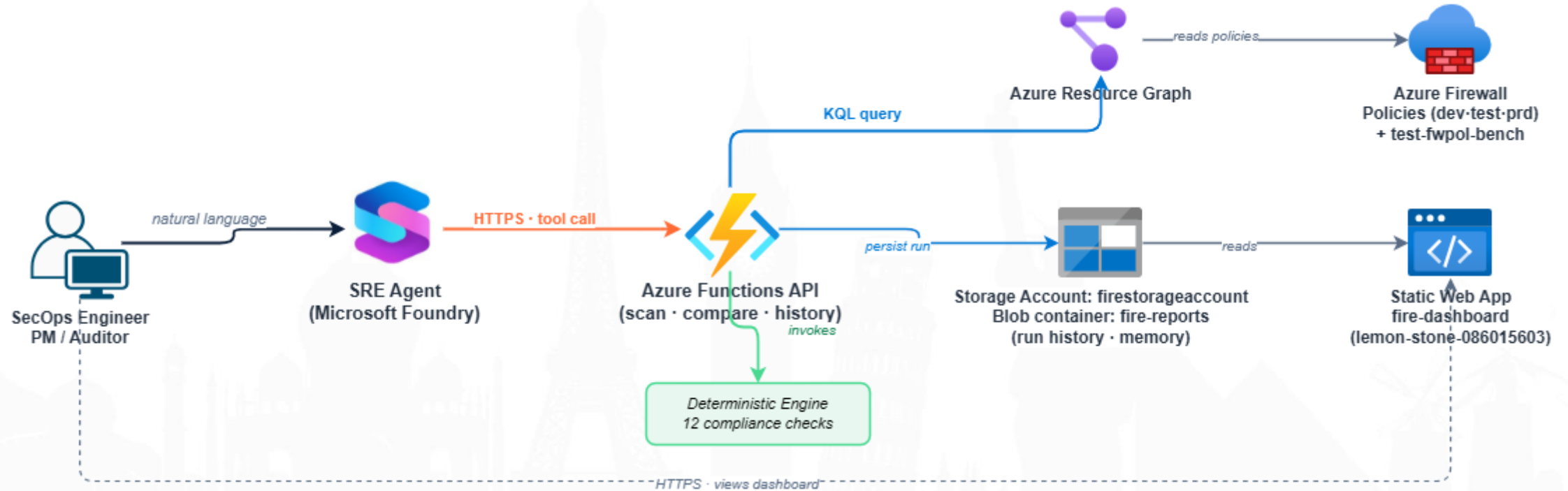


An LLM has no network stack, no file system, no executor. It generates tokens. Period.

- "GPT-4 calls my APIs"
→ false. The runtime parses the function-call output and makes the HTTP call
- "The model executes Python"
→ false. `ExecutePythonCode` is an external sandbox; the model only writes the code
- Security lives in the runtime
the model proposes, the runtime executes (or refuses)

An LLM doesn't call APIs: it WRITES API calls. Crucial difference for security, debugging, architecture.

Architecture



All serverless. No VMs, no dedicated CI. The skill is a Markdown file.

The deterministic engine — ground truth

12 Python checks running on ARG data

DangerousNetworkRules

WrongRuleType

RuleCollectionPriority

DnatRules

HardcodedIPs

DuplicateRules

DuplicateIPs

EmptyIpGroup

AzureFqdnServiceTag

ServiceTagPlacement

IpGroupNaming

PlatformRules

Output: list of findings {check, item, severity, details}.
This is the reference we measure the agent against.

What we use in FIRE — and why

- Hosted runtime
 - no infra — the agent lives in the portal, no containers, no CI*
- Skill = Markdown file with YAML frontmatter
 - name, description, tools — registered on the agent*
- Tool: ExecutePythonCode
 - the agent writes Python; the sandbox runs it (not the agent)*
- Telemetry + run history
 - every execution tracked, metrics and logs in App Insights*
- Natural conversation
 - "run the benchmark on prd" → the agent opens the skill*
- Underlying model
 - GPT-4 class via Foundry — abstracted, can evolve without breaking the skill*

Building Blocks of an Effective Agent

Key components that form the foundation of an agent system

1

System Prompt

Provides initial instructions and context for the agent

2

Orchestration

Coordinates actions and manages workflow between sub-agents

3

Tools

Enables interaction with external systems and tasks

4

Self-Correction

Identifies errors and adjusts actions for improvement

5

Memory

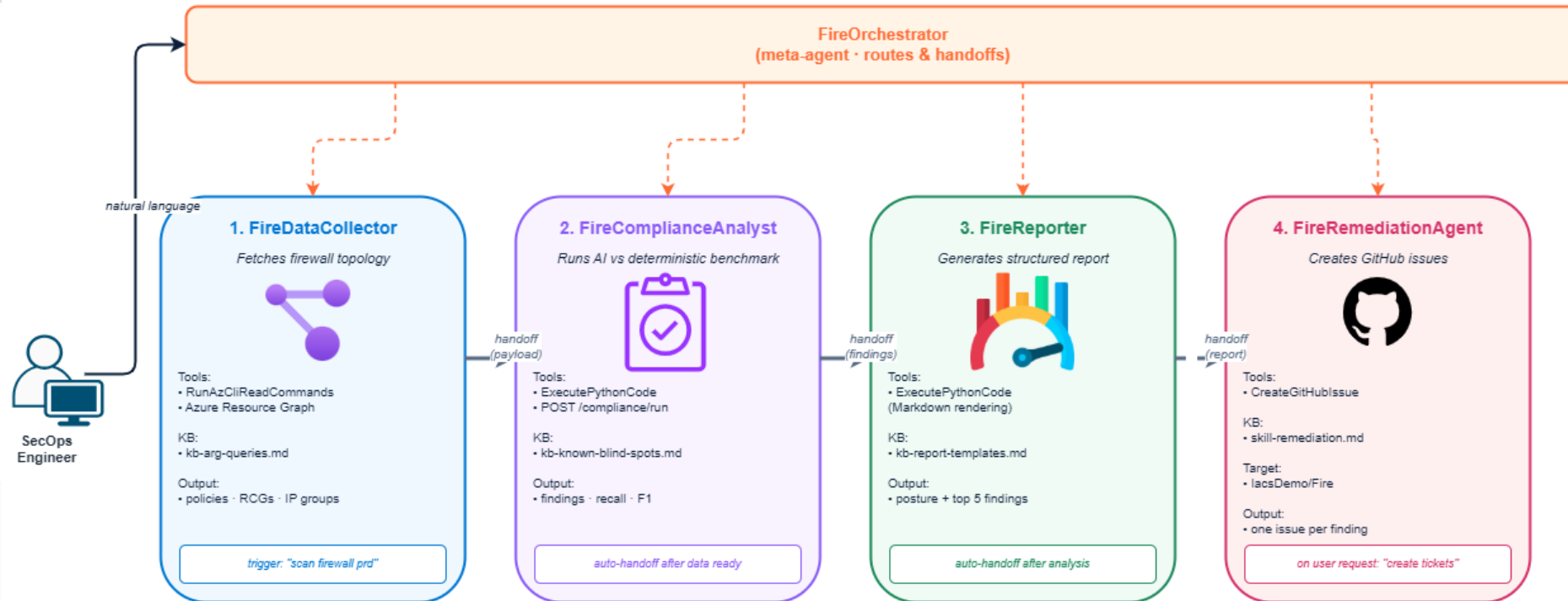
Stores past interactions to inform future decisions

Why Sub-Agents?

When a single agent is no longer the right unit of design

- Too many tools and custom instructions
to combine into a single agent without losing focus or accuracy
- 50+ prompts, 161 unique tools
scaling a monolithic agent past this point degrades reliability
- Reduce the potential scope of impact
from prompt changes — isolate failure modes per sub-agent
- Specialization beats generalization
each sub-agent owns a narrow domain with its own metrics

Sub-Agents



Why this matters

- ✓ Narrow scope per agent — each one is testable and debuggable in isolation
- ✓ Failure is contained — if Reporter breaks, Collector and Analyst outputs are already saved
- ✓ Prompt changes are local — tweak the Reporter template without touching the Collector
- ✓ Action requires explicit consent — handoff to Remediation only on user request, never automatic

Runtime: FireOrchestrator (Python · imperative) when called via API · or the SRE Agent meta-agent when used from chat. Same skills, two executors.

Inside the Agent — 4 Pillars

Delegation contracts, self-correction loops, grounded memory, and evaluation discipline

Sub-Agent Delegation

Meta-agent delegates tasks via a standard handoff contract:
intent · context · success criteria · timeout / fallbacks.

Self-Correction Loop

Reflect, verify, retry or rollback.
Critic / verifier checks · human-in-the-loop for low confidence.

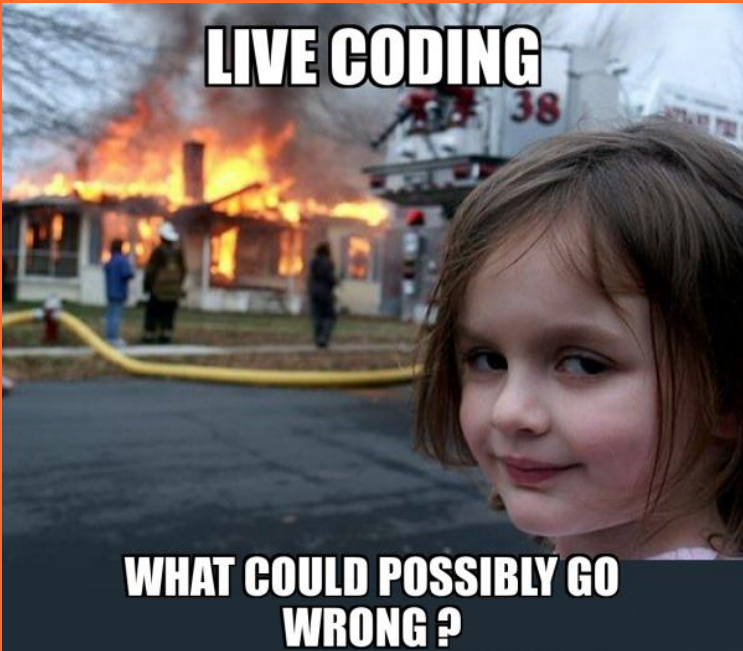
Memory & Grounding

Short-term thread state + long-term knowledge (KG / ADX).
Learn from mistakes · ground answers in domain data and logs.

Evaluation & Confidence

Unit tests · synthetic traces · canary A/B for prompt diffs.
Trace metrics: win-rate, resolution time, SAT/DSAT.
Measure prompt changes and their impact with confidence.

LIVE CODING #1



Anatomy of a skill

From vague prompt to mechanical recipe

Baseline — the naïve prompt reality

First skill: "find problems in the firewalls"

Recall

35%

Precision

51%

F1

42%

The AI missed 2 of 3. Made up rule names. Confused policy/rcg/rule levels.

Recall at 35%



When the PM asks 'how's the AI going?'

smiling outside, refactoring inside

Technique 1 — Pre-flattening

BEFORE

```
{
  "policies": [
    {
      "name": "fwpol-prd-eu",
      "location": "westeurope",
      "tier": "Premium",
      "ruleCollectionGroups": [
        {
          "name": "rcg-application",
          "priority": 100,
          "ruleCollections": [
            {
              "name": "coll-allow-web",
              "priority": 1000,
              "action": "Allow",
              "rules": [
                {
                  "name": "allow-github",
                  "ruleType": "ApplicationRule",
                  "sourceAddresses": ["10.1.0.0/16"],
                  "targetFqdns": ["*.github.com"],
                  "protocols": [{"protocolType": "Https", "port": 443}]
                }
              ]
            }
          ]
        }
      ]
    }
  ]
}
```

policies → ruleCollectionGroups → ruleCollections → rules → protocols

Technique 1 — Pre-flattening

BEFORE

```
{
  {
    "name": "allow-microsoft-update",
    "ruleType": "ApplicationRule",
    "sourceAddresses": ["10.1.0.0/16"],
    "targetFqdns": ["*.windowsupdate.com"],
    "protocols": [{"protocolType": "Https", "port": 443}]
  }
},
{
  "name": "coll-deny-tor",
  "priority": 200,
  "action": "Deny",
  "rules": [
    {
      "name": "block-tor-exits",
      "ruleType": "NetworkRule",
      "sourceAddresses": ["*"],
      "destinationAddresses": ["198.96.155.0/24"],
      "destinationPorts": ["*"],
      "ipProtocols": ["TCP"]
    }
  ]
}
```

Technique 1 — Pre-flattening

Force the agent to build a flat table BEFORE reasoning.

```
flat_rules = []
for rcg in rcgs:
    for rc in rcg.get("properties", {}).get("ruleCollections", []):
        for r in rc.get("rules", []):
            flat_rules.append({
                "policy": rcg["policyName"],
                "rcg": rcg["name"],
                "rc": rc["name"],
                "rc_priority": rc.get("priority"),
                "rule": r.get("name"),
                "ruleType": r.get("ruleType"),
                "srcAddr": r.get("sourceAddresses") or [],
                "dstAddr": r.get("destinationAddresses") or [],
                "dstPorts": r.get("destinationPorts") or [],
                "dstFqdns": r.get("destinationFqdns") or [],
            })
```

From hierarchical matryoshka to Excel-like rows. No more "which policy was this rule?".

Technique 2 — Pseudo-algorithms

12 imperative recipes — one per check. The AI runs them as pseudo-code.

```
### WrongRuleType - item = <rule>
for r in flat_rules:
    if r.ruleType == 'NetworkRule' and any(p in ['80','443','8080','8443']
                                           for p in r.dstPorts):
        emit WrongRuleType with item=r.rule
    if r.ruleType == 'ApplicationRule':
        for f in r.dstFqdns:
            if is_ip_or_cidr(f):
                emit WrongRuleType with item=r.rule

# Real examples the AI always missed:
#   AllowAzureHealthProbes (NetworkRule on 443,80)
#   AllowHTTPandHTTPS      (NetworkRule on 443,80)
```

Stop asking. Start specifying. From "find problems" to "run this function".

Technique 3 — Blind-spot checklist (primacy bias)

Top of the prompt weighs more. Put systematic mistakes there.

- > ⚠ Known AI blind spots (recall historically ~35%).
- > Treat these three families as MANDATORY checklists:
- >
- > 1. WrongRuleType – HTTP/HTTPS ports (80, 443, 8080, 8443)
inside NetworkRules
- > 2. RuleCollectionPriority – DNAT priority $\geq$ Network priority
in the same policy
- > 3. DnatRules – DNAT rules with sourceAddresses: ['*']

This trick alone: recall 35% → **~50%.**

Technique 4 — Few-shot with real data

No abstract examples. Only names that exist in the customer's tenant.

- Real per-check examples, drawn from the benchmark itself
- E.g. DangerousNetworkRules: AllowHTTPS (dst='*'), WebServerDNAT (src='*')
- E.g. DnatRules: DnatToInternalOpenSource (DNAT source='*')
- E.g. RuleCollectionPriority: fire-test-fwpol-bench (DNAT 2100 $\geq$ Network 500)
- Pattern matching beats abstract reasoning

The model generalizes better from 2-3 concrete examples than from a formal definition.

Technique 5 — Sanity filter (post-processing)

Python code run AFTER the model, on its own list.

```
# 1. normalize check names - must be one of 12 canonical
ai_findings = [
    {**f, "check": CANONICAL_CHECKS[norm(f["check"])]}
    for f in ai_findings if norm(f["check"]) in CANONICAL_CHECKS
]
# 2. item must literally exist in raw data
valid_items = {r["rule"] for r in flat_rules} | \
    {g["name"] for g in flat_ipg} | \
    {r["policy"] for r in flat_rules}
ai_findings = [f for f in ai_findings
    if f["item"] in valid_items or f["check"] == "DuplicateIPs"]
# 3. dedup on (check, item)
ai_findings = dedup(ai_findings)
```

```
[
    { "check": "DangerousNetworkRules", "item":
      "fwpol-prd/rcg-net/coll-allow/rdp-rule",
      "severity": "high" },
    { "check": "WrongRuleType", "item":
      "fwpol-prd/rcg-app/coll-web/allow-github",
      "severity": "medium" },
    { "check": "Wrong Rule Type", "item":
      "fwpol-prd/rcg-app/coll-web/allow-github",
      "severity": "medium" },
    { "check": "wrong_rule_type", "item":
      "fwpol-prd/rcg-app/coll-web/allow-github",
      "severity": "high" },
    { "check": "DangerousRules", "item":
      "fwpol-prd/rcg-net/coll-allow/ssh-
rule", "severity": "high" },
    { "check": "HardcodedIPs", "item":
      "fwpol-prd/rcg-net/coll-deleted/old-rule",
      "severity": "low" }
]
```

Not ML. It's defense-in-depth against small model drifts.

Final result — same model, same skill

+53 points of recall · +30 points of F1

Recall

88%

Precision

60%

F1

72%

Zero training. Zero GPU. Zero extra cost. Pure data-driven skill engineering.

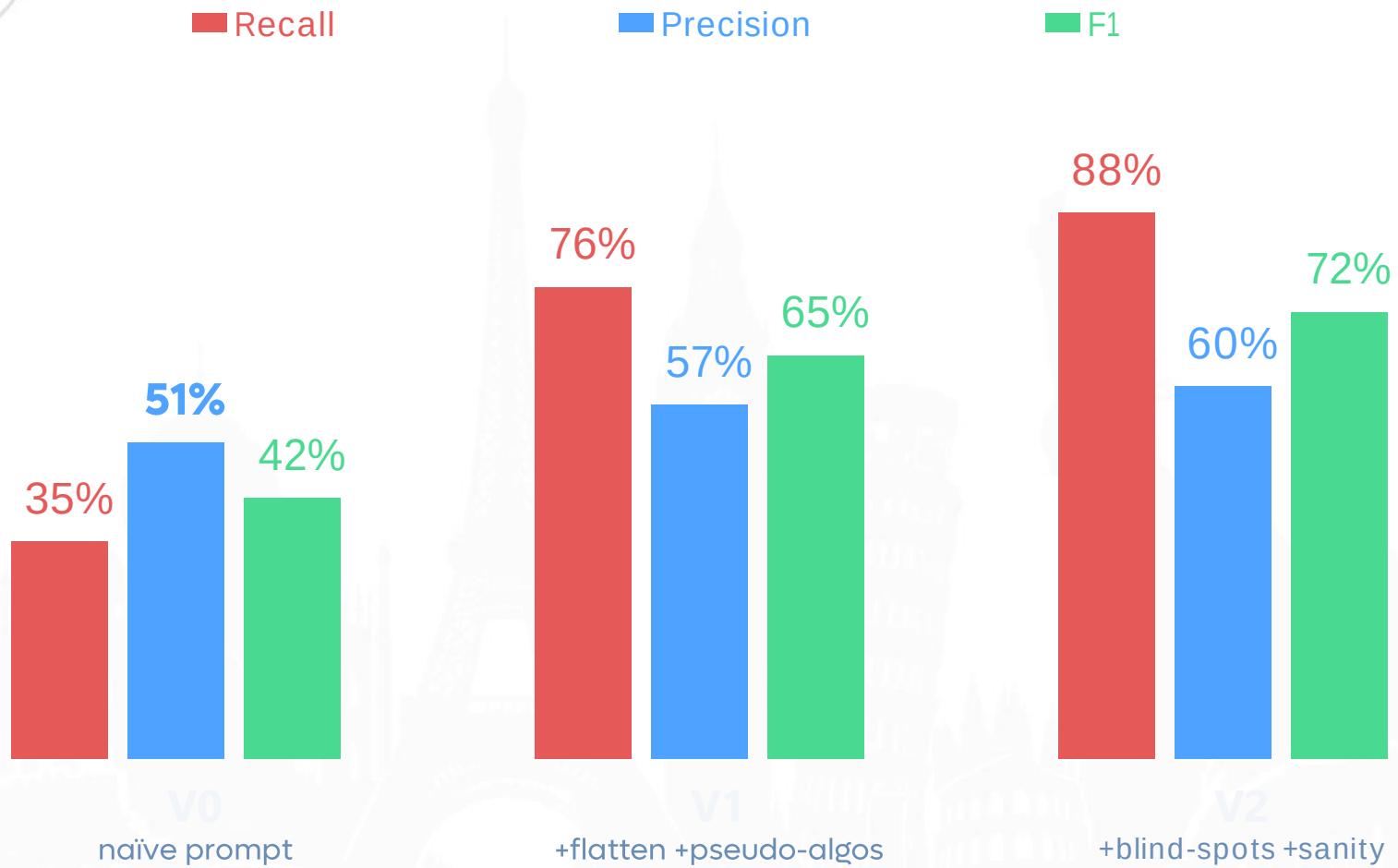
+53 points of recall



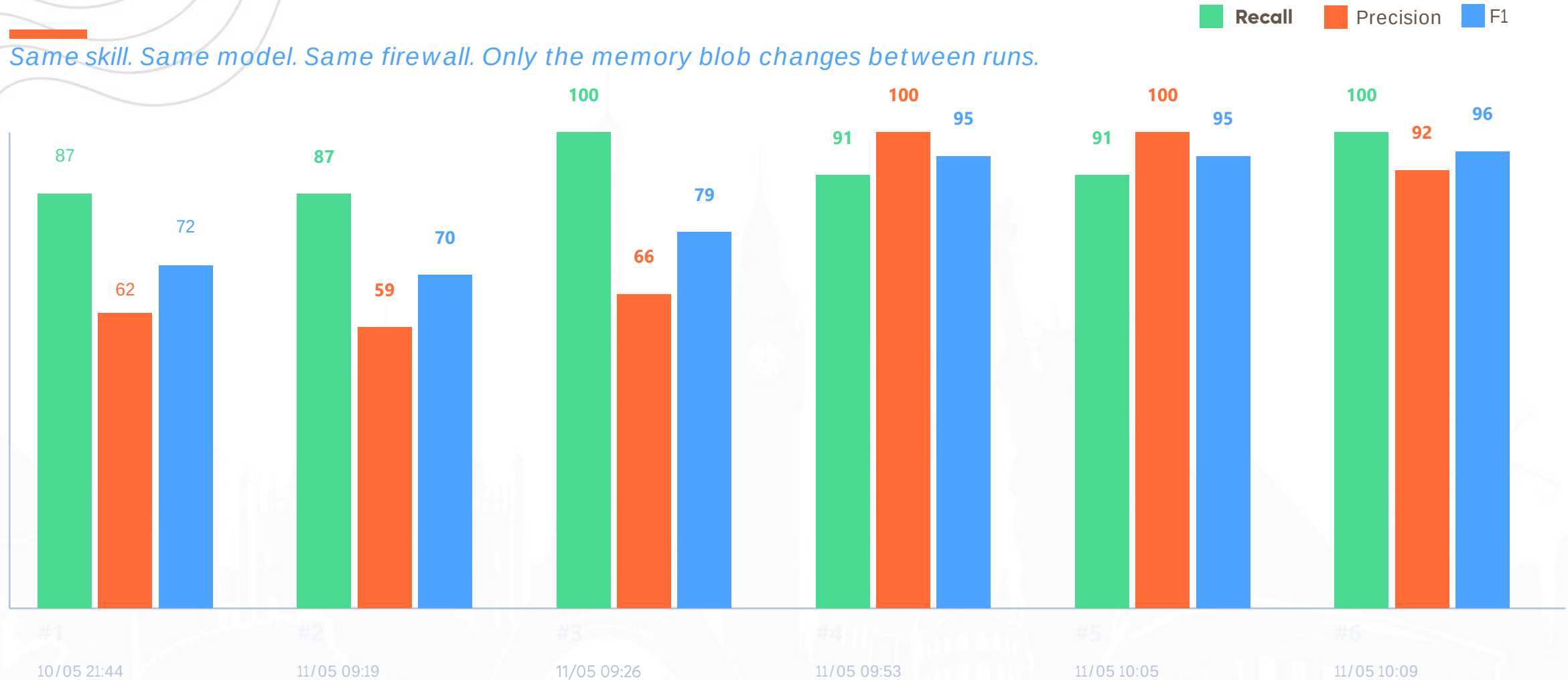
Computing recall improvements from prompt engineering

Zero training. Zero GPU. Just well-written Markdown.

Metrics evolution



Self-Correction in Action — 6 real runs



F1: 72 → 96 in 6 iterations • 30 minutes • no retraining • 4 KB JSON blob

Memory stores check categories only — never item names. Otherwise: bench-cheating.

LIVE CODING #2

LIVE CODING

**WHAT COULD POSSIBLY GO
WRONG ?**

Run the agent

Live benchmark on real Azure Firewall data

Lessons learned



Always measure

Without a benchmark you can't tell good prompts from cool ones



Pre-processing > prompting

Data shape is 50% of the work



Pseudo-code beats natural language

For structured, repetitive tasks



Primacy bias is real

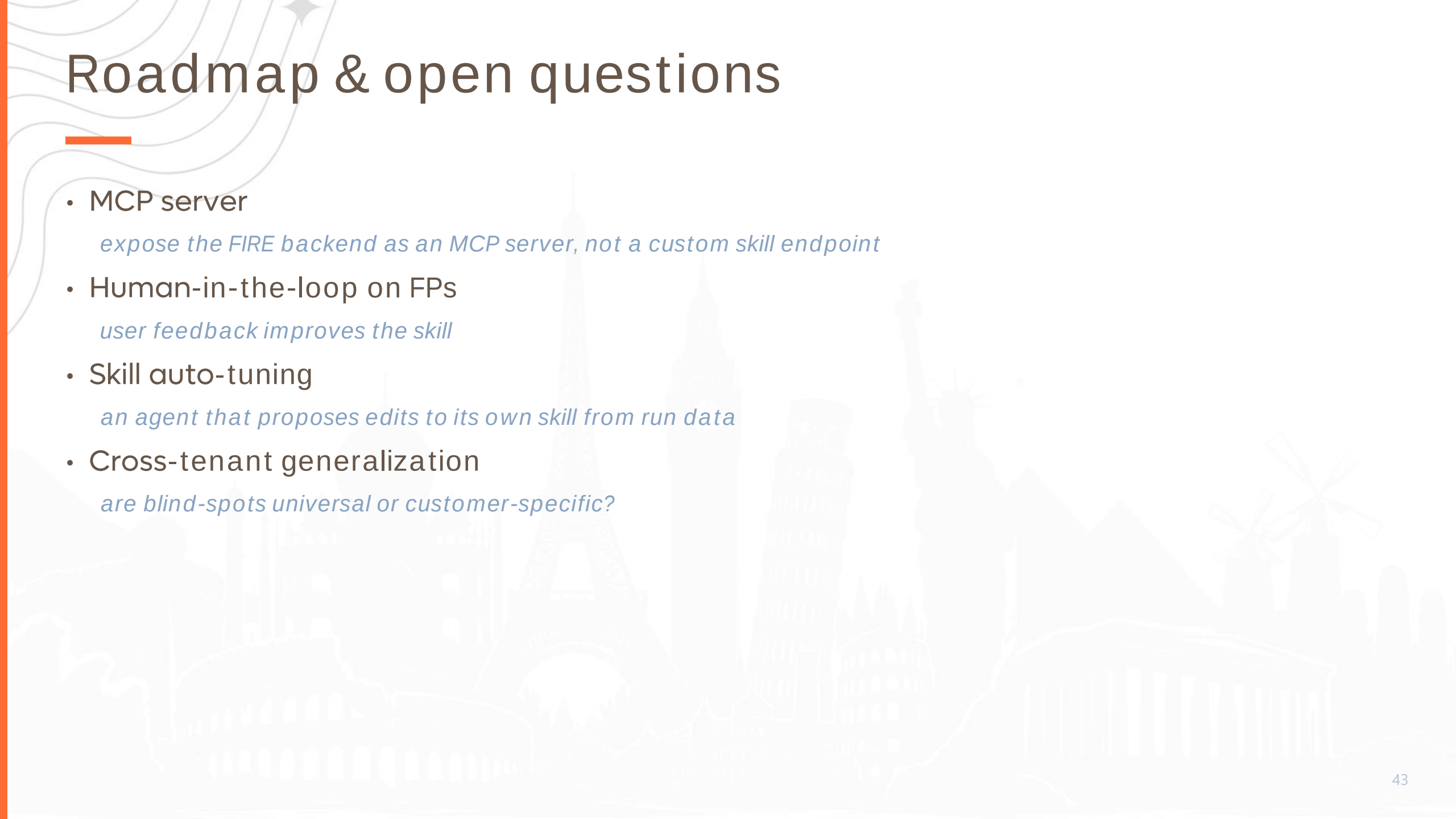
Always put critical cases at the top of the prompt



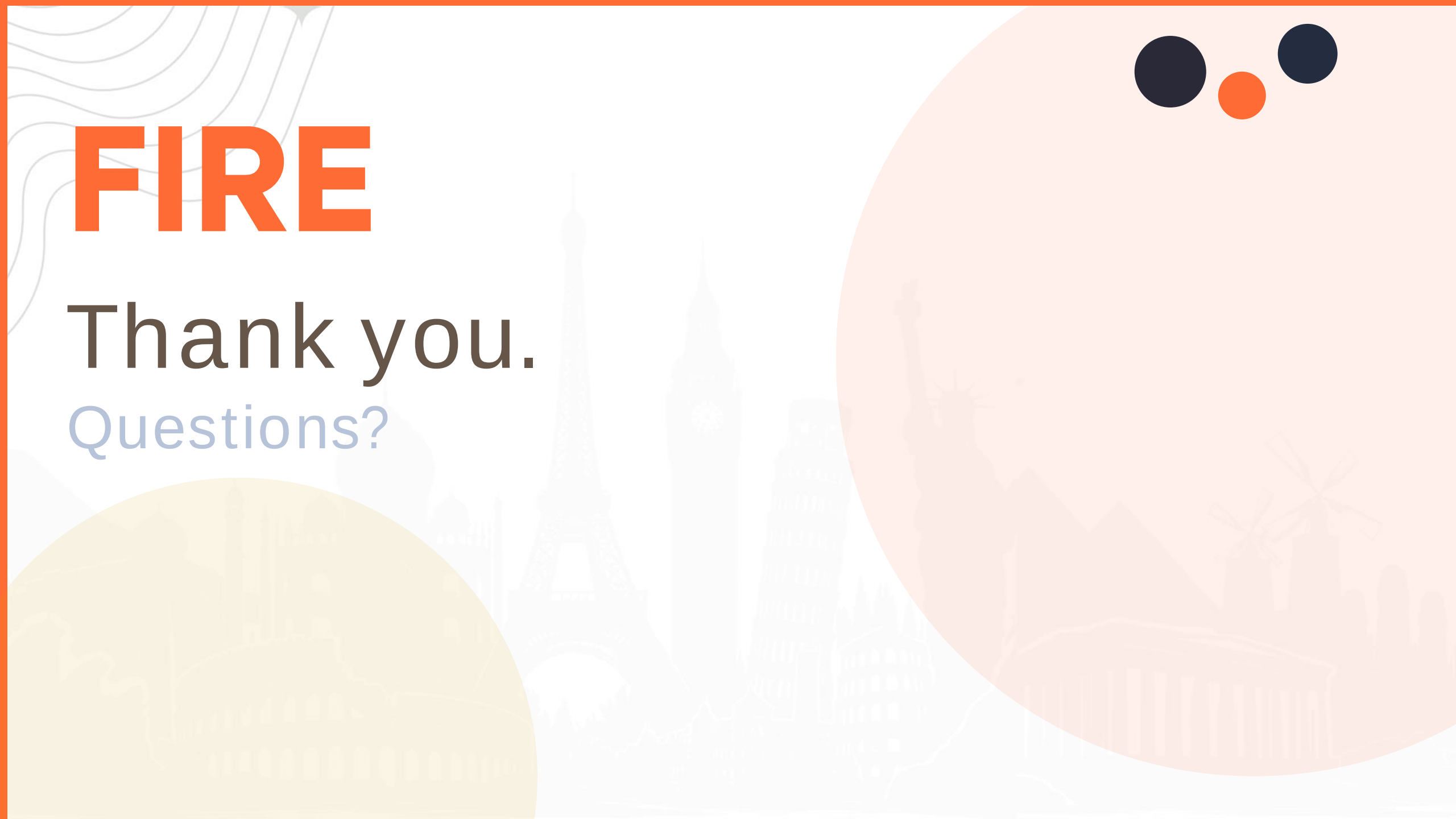
Few-shot from your real dataset

Don't invent examples — copy them from the data

Roadmap & open questions



- MCP server
 - expose the FIRE backend as an MCP server, not a custom skill endpoint*
- Human-in-the-loop on FPs
 - user feedback improves the skill*
- Skill auto-tuning
 - an agent that proposes edits to its own skill from run data*
- Cross-tenant generalization
 - are blind-spots universal or customer-specific?*

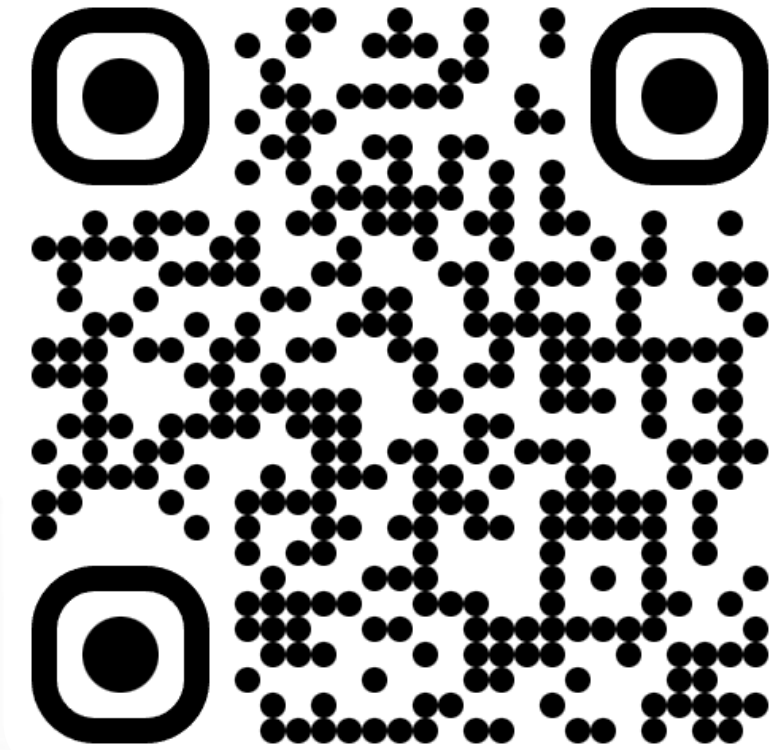


FIRE

Thank you.

Questions?

Join the AI Conversation on Discord



gaic.io/discord